

# NGÔN NGỮ TRUY VẤN SQL

# Outline

- Giới thiệu
- Định nghĩa dữ liệu
- Cập nhật dữ liệu
- Truy vấn dữ liệu

# Giới thiệu

- SQL = **S**tructured **Q**uery **L**anguage
- Gồm:
  - Định nghĩa dữ liệu
  - Thao tác dữ liệu
  - Định nghĩa khung nhìn
  - Phân quyền và bảo mật

# Định nghĩa dữ liệu

- Gồm các lệnh
  - CREATE / DROP DATABASE
  - CREATE / DROP / ALTER TABLE
  - CREATE / DROP DOMAIN

# Tạo bảng

- CREATE TABLE <tên bảng> (  
    <tên thuộc tính> <kiểu dữ liệu> [<RBTV>],  
    <tên thuộc tính> <kiểu dữ liệu> [<RBTV>],  
    ...  
    [<RBTV>]  
)
- RBTV
  - NOT NULL
  - NULL
  - UNIQUE
  - DEFAULT
  - PRIMARY KEY
  - FOREIGN KEY / REFERENCES
  - CHECK

# Tạo bảng

- Ví dụ

```
create table NHANVIEN (  
    Ho          varchar(15)    not null,  
    Dem         varchar(20),  
    Ten         varchar(15)    not null,  
    MaNV        char(9)        not null,  
    Ngsinh      date,  
    Dchi        varchar(30),  
    GTinh       char,  
    Luong       decimal(10,2),  
    MaGSat      char(9),  
    MaPhong     int            not null  
)
```

# Tạo bảng

- Đặt tên cho RBTV
  - CONSTRAINT <tên RBTV> <RBTV>
- Ví dụ

```
create table PHONGBAN (  
    Ten          varchar(15)    unique,  
    MaPB         int            not null,  
    TrPhong      varchar(9)     not null,  
    NgNhanChuc   date,  
    constraint PB_PK primary key (MaPB),  
    constraint PB_TrPh foreign key (TrPhong) references  
    NHANVIEN (MaNV) on delete set null on update cascade  
)
```

# Sửa bảng

- ALTER TABLE <tên bảng> ADD COLUMN <tên thuộc tính> <kiểu dữ liệu> [<RBTV>]
  - Ví dụ  
alter table NHANVIEN add column DThoi char(10)
- ALTER TABLE <tên bảng> DROP COLUMN <tên thuộc tính>
  - Ví dụ  
alter table NHANVIEN drop column DThoi
- ALTER TABLE <tên bảng> ALTER COLUMN <tên thuộc tính> <kiểu dữ liệu mới>
  - Ví dụ  
alter table NHANVIEN alter column GTinh int

# Sửa bảng

- ALTER TABLE <tên bảng> ADD CONSTRAINT <tên RBTV> <RBTV>

- Ví dụ

```
alter table NHANVIEN add constraint NV_Luong check  
(Luong > 0)
```

- ALTER TABLE <tên bảng> DROP CONSTRAINT <tên RBTV>

- Ví dụ

```
alter table NHANVIEN drop constraint NV_Luong
```

# Tạo và xóa miền trị

- `CREATE DOMAIN <tên kiểu dữ liệu mới> AS <kiểu dữ liệu cơ bản>`
  - Ví dụ  
`create domain Diachi as varchar(100)`
- `DROP DOMAIN <tên kiểu dữ liệu>`

# Cập nhật dữ liệu

## ▪ Chèn dữ liệu

### • Chèn từng dòng

- INSERT INTO <tên bảng> [(<danh sách thuộc tính>)]  
VALUES (<danh sách giá trị>)

- Ví dụ

- + insert into DUAN (TenDA, MaDA, Diadiem, Phong) values  
( 'San pham X',1,'Tan Binh',5)

- + insert into DUAN values ('San pham X',1,'Tan Binh',5)

### • Chèn nhiều dòng

- INSERT INTO <tên bảng> [(<danh sách thuộc tính>)]  
<lệnh truy vấn>

# Xóa dữ liệu

- DELETE FROM <tên bảng> [WHERE <điều kiện>]
- Ví dụ
  - delete from NHANVIEN where MaPhong = 5
  - delete from NHANVIEN

# Sửa dữ liệu

- UPDATE <tên bảng>  
SET <tên thuộc tính> = <giá trị mới>,  
    <tên thuộc tính> = <giá trị mới>,  
    ...  
[WHERE <điều kiện>]
- Ví dụ
  - update DUAN  
    set Diadiem = 'Hanoi', Phong = 4  
    where Maso=10
  - update NHANVIEN  
    set Luong = 1.1 \* Luong

# Truy vấn dữ liệu

- Cú pháp
  - SELECT <dstt>  
FROM <dsb>  
[WHERE <đk>]  
[GROUP BY <dstt nhóm>]  
[HAVING <đk nhóm>]  
[ORDER BY <dstt sắp xếp>]
- Kết quả của lệnh truy vấn là một bảng.
- Bảng trong SQL có thể chứa các bộ trùng nhau.
- Trong đó
  - <dstt>: danh sách thuộc tính của quan hệ kết quả.
  - <dsb>: danh sách bảng liên quan đến câu truy vấn.
  - <đk>: điều kiện để chọn hoặc ghép các bộ.
  - <dstt nhóm>: danh sách thuộc tính để nhóm các bộ.
  - <đk nhóm>: điều kiện chọn các nhóm.
  - <dstt sắp xếp>: danh sách các thuộc tính và thứ tự sắp xếp tương ứng.

# Truy vấn dữ liệu và đại số quan hệ

- $\pi_{\langle \text{dstt} \rangle}(R)$   
select <dstt>  
from R

- $\sigma_{\langle \text{đkc} \rangle}(R)$   
select \*  
from R  
where <đkc>

- $R \times S$   
select \*  
from R, S

- $R \bowtie_{\langle \text{đkk} \rangle} S$   
select \*  
from R, S  
where <đkk>

- $\pi_{\langle \text{dstt} \rangle}(\sigma_{\langle \text{đkc} \rangle}(R))$   
select <dstt>  
from R  
where <đkc>

# Truy vấn dữ liệu và đại số quan hệ

- Tìm các nhân viên làm việc trong phòng số 4.
  - $\sigma_{MaPB = 4}(NHANVIEN)$
  - `select *`  
`from NHANVIEN`  
`where MaPB = 4`
- Cho biết họ, tên, giới tính và mức lương của các nhân viên.
  - $\pi_{Ho, Ten, GTinh, Luong}(NHANVIEN)$
  - `select Ho, Ten, GTinh, Luong`  
`from NHANVIEN`
- Cho biết tên các trưởng phòng
  - $\pi_{Ho, Ten}(PHONGBAN \bowtie_{TrPhg=MaNV} NHANVIEN)$
  - `select Ho, Ten`  
`from NHANVIEN, PHONGBAN`  
`where TrPhg = MaNV`

# Phép toán tập hợp

- SQL hỗ trợ các phép toán
  - UNION (Hội)
  - EXCEPT (Hiệu).
  - INTERSECT (Giao).
  - Đặc điểm
    - Các dòng giống nhau bị loại trong bảng kết quả.
    - Các bảng tham gia phép toán phải có tính khả hợp
- Giữ lại các dòng giống nhau
  - UNION ALL
  - EXCEPT ALL
  - INTERSECT ALL
- (SELECT <dstt> FROM <dsb>  
WHERE <đk>)  
UNION [ALL]  
(SELECT <dstt> FROM <dsb>  
WHERE <đk>)
- (SELECT <dstt> FROM <dsb>  
WHERE <đk>)  
EXCEPT [ALL]  
(SELECT <dstt> FROM <dsb>  
WHERE <đk>)
- (SELECT <dstt> FROM <dsb>  
WHERE <đk>)  
INTERSECT [ALL]  
(SELECT <dstt> FROM <dsb>  
WHERE <đk>)

# Bí danh - Alias

- Đặt tên trong mệnh đề SELECT:

```
select count(*) as 'Tong so nhan vien'  
from NHANVIEN
```

- Đặt tên trong mệnh đề FROM:

- Đặt tên cho bảng

```
select NV.Ho, NV.Ten, PB.Ten  
from NHANVIEN as NV, PHONGBAN as PB  
where NV.MaPB = PB.MaPB
```

- Đặt tên cho bảng và thuộc tính

```
select *  
from PHONGBAN AS PB(Ten, Maso, MaTrPhg, Ngay)
```

# Phép toán số học

- $+$ ,  $-$ ,  $*$ ,  $/$  có thể áp dụng cho các *giá trị số* trong mệnh đề SELECT

```
select 1.1 * SUM(Luong) as 'Luong moi'  
from NHANVIEN
```

| NHANVIEN | ... | Luong | ... |
|----------|-----|-------|-----|
|          |     | 40000 |     |
|          |     | 30000 |     |
|          |     | 38000 |     |
|          |     | 25000 |     |

| Luong moi |
|-----------|
| 44000     |
| 33000     |
| 41800     |
| 27500     |

- $+$ ,  $-$  có thể áp dụng cho các *giá trị kiểu ngày giờ*  

```
select NgNhanChuc + 150 as 'Cong Ngay'  
from PHONGBAN
```

# Phép toán so sánh và luận lý

- Dùng trong mệnh đề WHERE hoặc HAVING để xây dựng các điều kiện chọn và điều kiện kết.
  - =, <, ≤, >, ≥, <>
  - BETWEEN <giá trị> AND <giá trị>
  - AND, OR, NOT
- Tìm các nhân viên phòng số 5 có lương giữa 30.000 và 40.000
  - ```
select *  
from NHANVIEN  
where (Luong >= 30000) and (Luong <= 40000) and Phg = 5
```
  - ```
select *  
from NHANVIEN  
where (Luong between 30000 and 40000) and Phg = 5
```

# Distinct

- `select Luong  
from NHANVIEN`

| Luong |
|-------|
| 40000 |
| 30000 |
| 38000 |
| 25000 |
| 43000 |
| 25000 |
| 25000 |

- SQL không tự động loại các bộ trùng nhau
  - Tốn thời gian so sánh và sắp xếp.
  - Sử dụng cho các truy vấn thống kê
- `select distinct Luong  
from NHANVIEN`

| Luong |
|-------|
| 40000 |
| 30000 |
| 38000 |
| 25000 |
| 43000 |

# Các hàm tập hợp

- SQL cung cấp 5 hàm tập hợp:
  - SUM(<tên thuộc tính>) - tính tổng các giá trị của thuộc tính
  - MAX(<tên thuộc tính>) - tìm giá trị lớn nhất của thuộc tính
  - MIN(<tên thuộc tính>) - tìm giá trị nhỏ nhất của thuộc tính
  - AVG(<tên thuộc tính>) - tính giá trị trung bình của thuộc tính
  - COUNT(\*) - đếm số dòng của bảng
  - COUNT(<tên thuộc tính>) - đếm các giá trị khác null của thuộc tính
- Ví dụ

```
select sum(Luong) AS TongLuong,
       max(Luong) AS LuongCaonhat,
       min(Luong) AS LuongThapnhat,
       avg(Luong) AS LuongTB
from NHANVIEN
```
- Chỉ được xuất hiện trong mệnh đề SELECT hoặc HAVING

# Gom nhóm

- GROUP BY - HAVING

```
SELECT <dstt nhóm> [, <dshth>]  
FROM <dsb>  
[WHERE <đk>]  
GROUP BY <dstt nhóm>  
[HAVING <đk nhóm>]
```

- Trong đó

- <dstt nhóm>: danh sách thuộc tính gom nhóm
- <dshth>: danh sách các hàm tập hợp.
- <đk>: điều kiện chọn hoặc điều kiện kết.
- <đk nhóm>: điều kiện lựa chọn các nhóm.

- Chú ý

- WHERE được thực hiện trước GROUP BY.
- HAVING chỉ xuất hiện khi có GROUP BY

# Gom nhóm

- Với mỗi phòng, cho biết số dự án phòng đó điều phối.

```
select Phong, count(MaDA) as 'So du an'  
from DUAN  
group by Phong
```

| DUAN | TenDA       | MaDA | Diadiem   | Phong |
|------|-------------|------|-----------|-------|
|      | Sản phẩm X  | 1    | Tân Bình  | 5     |
|      | Sản phẩm Y  | 2    | Thủ Đức   | 5     |
|      | Sản phẩm Z  | 3    | Phú Nhuận | 5     |
|      | Tin học hóa | 10   | Gò Vấp    | 4     |
|      | Phúc lợi    | 30   | Gò Vấp    | 4     |
|      | Tái tổ chức | 20   | Phú Nhuận | 1     |

| Phong | So du an |
|-------|----------|
| 5     | 3        |
| 4     | 2        |
| 1     | 1        |

# Sắp xếp kết quả

## ▪ ORDER BY

- SELECT <dstt>  
FROM <dsb>  
[WHERE <đk>]  
ORDER BY <dstt sắp xếp>
- <dstt sắp xếp>: danh sách các cặp (tên thuộc tính, thứ tự sắp xếp).
- Thứ tự:
  - ASC - tăng dần.
  - DESC - giảm dần.
  - Mặc định là ASC.

# Sắp xếp kết quả

- Với mỗi nhân viên, cho biết mã nhân viên và mã dự án mà nhân viên đó tham gia. Sắp xếp kết quả theo thứ tự tăng dần của mã nhân viên và giảm dần của mã dự án.
  - `select MaNV, MaDA`  
`from THAMGIA`  
`order by MaNV, MaDA desc`

| MaNV      | MaDA |
|-----------|------|
| 123456789 | 2    |
| 123456789 | 1    |
| 333445555 | 20   |
| 333445555 | 10   |
| 333445555 | 3    |
| 333445555 | 2    |
| ...       |      |

# So sánh với NULL

- SQL cung cấp 2 phép toán
  - IS NULL.
  - IS NOT NULL.

```
delete from tblsinhvien  
where masv is null;
```

# Truy vấn lồng nhau

- Truy vấn sử dụng các giá trị của truy vấn khác trong điều kiện so sánh.

Truy vấn cha {  
SELECT <dstt>  
FROM <dsb>  
WHERE <so sánh tập hợp> (  
    SELECT <dstt>  
    FROM <dsb>  
    WHERE <đk> ) } Truy vấn con

- Chỉ xuất hiện trong mệnh đề WHERE.

# Truy vấn lồng nhau

## ▪ Phép toán

- IN - kiểm tra sự tồn tại của một giá trị trong một tập hợp.
- ALL - so sánh một giá trị với tất cả các giá trị của tập hợp.
- ANY - so sánh một giá trị với một giá trị nào đó của tập hợp.
- ALL, ANY được kết hợp với các phép toán so sánh {=, <, ≤, >, ≥, <>}.
- EXISTS - kiểm tra sự tồn tại của kết quả của một câu truy vấn.

## ▪ Cú pháp

- <tên thuộc tính> IN <truy vấn con>
- <tên thuộc tính> <phép toán so sánh> ALL <truy vấn con>
- <tên thuộc tính> <phép toán so sánh> ANY <truy vấn con>
- EXISTS <truy vấn con>

# Truy vấn lồng nhau

- Tìm các nhân viên của phòng số 2 và 5.

```
select *  
from NHANVIEN  
where MaPB in (2, 5)
```

Tập hợp tường minh

- Tìm các nhân viên của phòng 'Nghiên cứu'.

```
select *  
from NHANVIEN  
where MaPB = (  
    select MaPB  
    from PHONGBAN  
    where TenPB =  
        'Nghiên cứu')
```

Truy vấn con vô hướng

# Truy vấn lồng nhau

- Cho biết tên nhân viên có mức lương lớn hơn lương của các nhân viên phòng số 5.
- Cho biết các nhân viên không tham gia các dự án mà phòng số 5 quản lý.

```
select *  
from NHANVIEN  
where Luong > ALL (  
  select Luong  
  from NHANVIEN  
  where MaPB = 5)
```

```
select MaNV  
from THAMGIA  
where MaDA not in (  
  select MaNV  
  from DUAN  
  where Phg = 5)
```

# Truy vấn lồng phân cấp

- Mệnh đề WHERE của truy vấn con không tham chiếu đến thuộc tính của các bảng trong mệnh đề FROM của truy vấn cha.
- Truy vấn con được thực hiện trước truy vấn cha.
- Ví dụ

```
select *  
from NHANVIEN  
where Luong > ALL (  
    select Luong  
    from NHANVIEN  
    where MaPB = 5)
```

# Truy vấn lồng phân cấp

| NHANVIEN | MaNV      | ... | Luong |
|----------|-----------|-----|-------|
|          | 888665555 |     | 55000 |
|          | 333445555 |     | 40000 |
|          | 123456789 |     | 30000 |
|          | 666884444 |     | 38000 |
|          | 453453453 |     | 25000 |
|          | 987654321 |     | 43000 |
|          | 999887777 |     | 25000 |
|          | 987987987 |     | 25000 |

> ALL

| Luong |
|-------|
| 40000 |
| 30000 |
| 38000 |
| 25000 |

| MaNV      | ... | Luong |
|-----------|-----|-------|
| 888665555 |     | 55000 |
| 987654321 |     | 43000 |

# Truy vấn lồng tương quan

- Mệnh đề WHERE của truy vấn con tham chiếu đến thuộc tính của các bảng trong mệnh đề FROM của truy vấn cha.
- Truy vấn con được thực hiện nhiều lần, mỗi lần ứng với một bộ của truy vấn cha.
- Tìm các nhân viên không có thân nhân nào

```
select *  
from NHANVIEN as NV  
where not exists (  
    select *  
    from THANNHAN as TN  
    where TN.MaNV = NV.MaNV)
```

# Kết nối trong SQL

- JOIN, INNER JOIN
  - Dùng kết nối hai bảng trong mệnh đề FROM.
  - SELECT <dstt>  
FROM (<bảng 1> JOIN <bảng 2> ON <đkk>)
- Các phép kết mở rộng:
  - LEFT OUTER JOIN, LEFT JOIN.
  - RIGHT OUTER JOIN, RIGHT JOIN.
  - FULL OUTER JOIN, FULL JOIN.

# Kết nối trong SQL

- Cho biết tên các nhân viên của phòng 'Nghiên cứu'.  

```
select NV.Ho, NV.Dem,  
       NV.Ten  
from (NHANVIEN as NV join  
      PHONGBAN as PB on  
      NV.MaPB =  
      PB.MaPB)  
where PB.Ten = 'Nghiên cứu'
```
- Các phép kết có thể lồng nhau  

```
select DA.MaDA, PB.MaPB,  
       NV.Ten  
from ((DUAN as DA join  
       PHONGBAN as PB  
       on DA.Phong =  
       PB.Maso)  
join NHANVIEN on  
      MaNQL = NV.MaNV)  
where Diadiem = 'TpHCM'
```

# Kết nối trong SQL

– CREATE TEST TABLES

```
CREATE TABLE Table1 (ID [int], Code [varchar] (10));  
GO
```

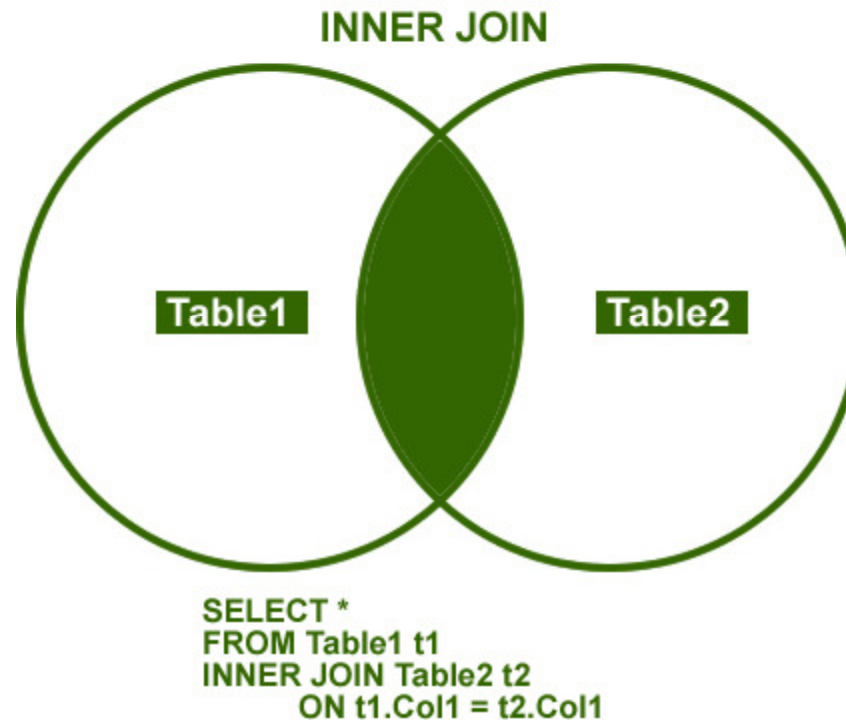
```
CREATE TABLE Table2 (ID [int], Amount [int]);  
GO
```

– INSERT DUMMY DATA

```
INSERT INTO Table1 VALUES  
(1, 'AAA'), (2, 'BBB'), (3, 'CCC')  
GO
```

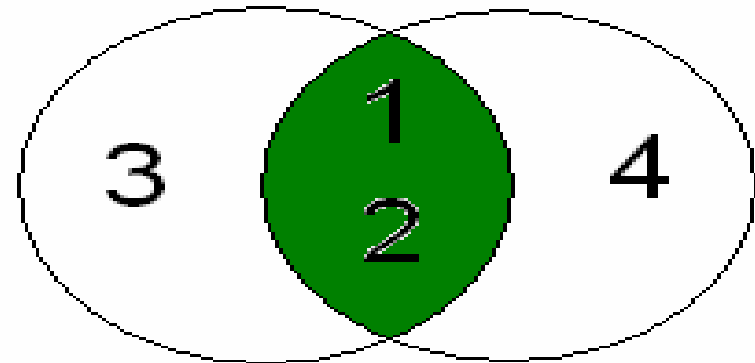
```
INSERT INTO Table2 VALUES  
(1, 50), (2, 30), (4, 40)  
GO
```

# Inner Join



# Inner join

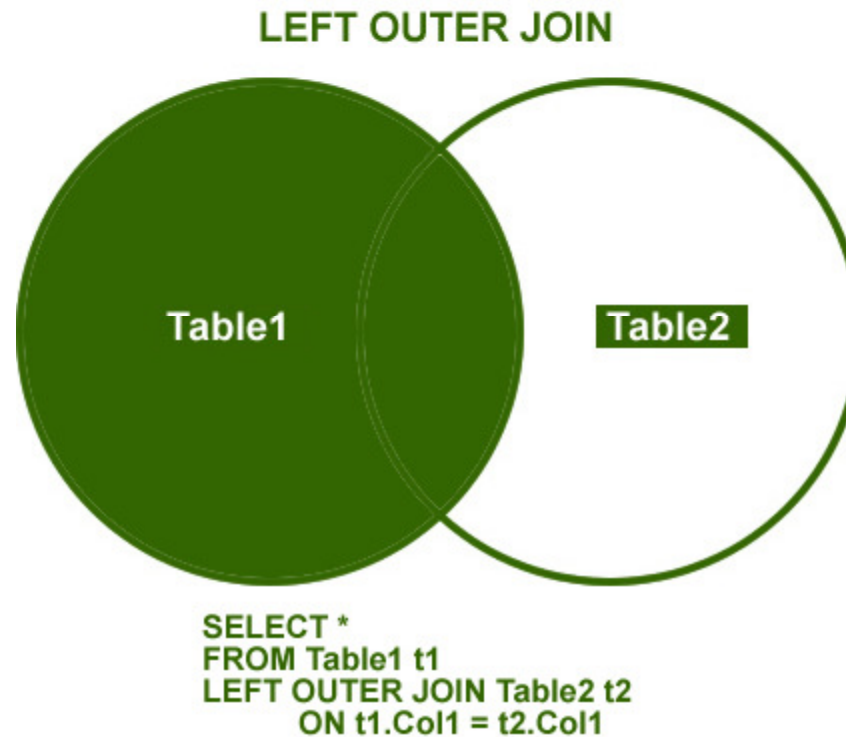
```
SELECT *  
FROM Table1 A  
INNER JOIN Table2 B  
    ON A.ID = B.ID  
GO
```



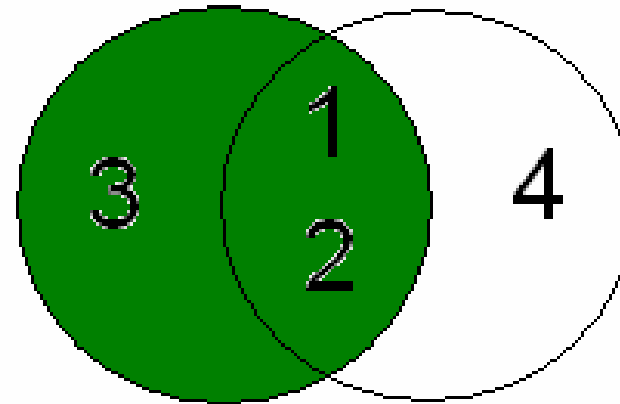
```
SELECT *  
FROM Table1 A  
INNER JOIN Table2 B  
    ON A.ID = B.ID  
GO
```

| <                |    |      |    |        |
|------------------|----|------|----|--------|
| Results Messages |    |      |    |        |
|                  | ID | Code | ID | Amount |
| 1                | 1  | AAA  | 1  | 50     |
| 2                | 2  | BBB  | 2  | 30     |

# Left outer join



# Left outer join



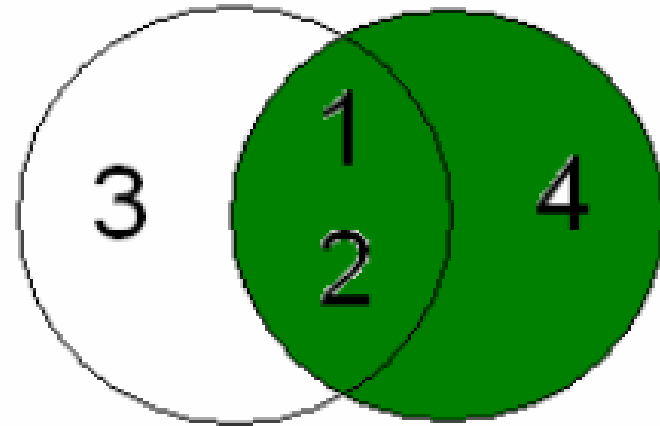
```
SELECT *  
FROM Table1 A  
LEFT OUTER JOIN Table2 B  
    ON A.ID = B.ID  
GO
```

|   | ID | Code | ID   | Amount |
|---|----|------|------|--------|
| 1 | 1  | AAA  | 1    | 50     |
| 2 | 2  | BBB  | 2    | 30     |
| 3 | 3  | CCC  | NULL | NULL   |

# Right outer join



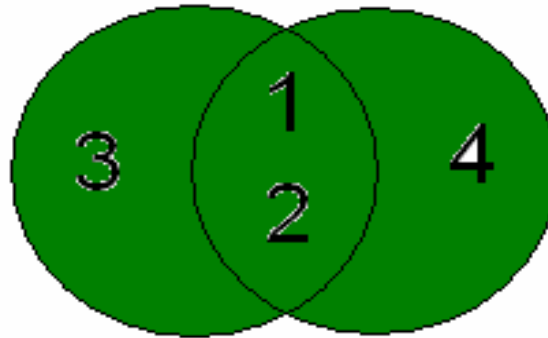
# Right outer join



```
SELECT *  
FROM Table1 A  
RIGHT OUTER JOIN Table2 B  
ON A.ID = B.ID  
GO
```

| <                |      |      |      |        |
|------------------|------|------|------|--------|
| Results Messages |      |      |      |        |
|                  | ID   | Code | I... | Amount |
| 1                | 1    | AAA  | 1    | 50     |
| 2                | 2    | BBB  | 2    | 30     |
| 3                | NULL | NULL | 4    | 40     |

# Full outer join

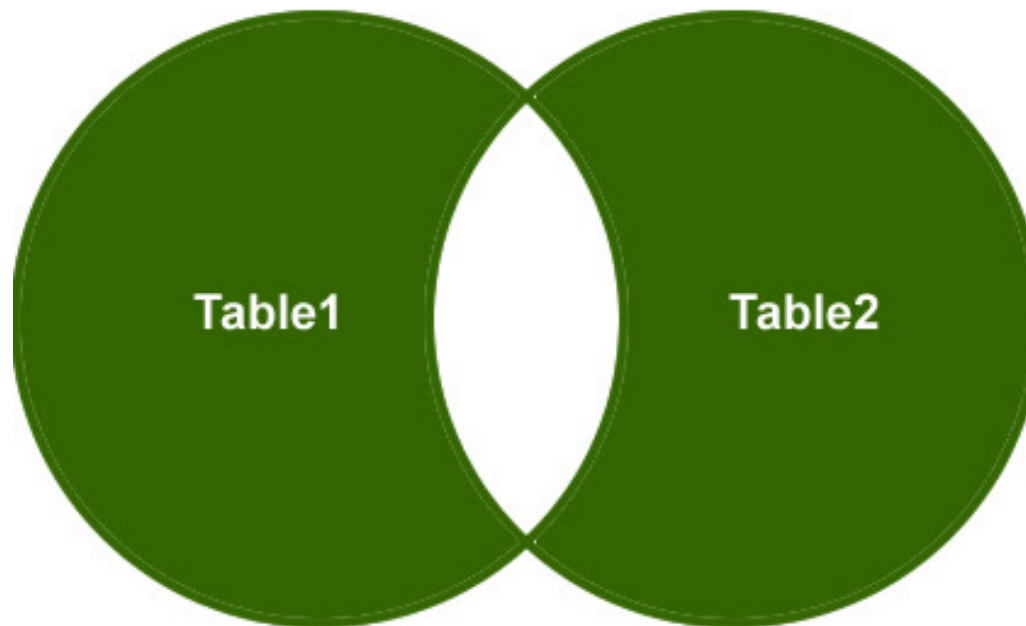


```
SELECT *  
FROM Table1 A  
FULL OUTER JOIN Table2 B  
ON A.ID = B.ID  
GO
```

| <                |      |      |      |        |
|------------------|------|------|------|--------|
| Results Messages |      |      |      |        |
|                  | ID   | Code | ID   | Amount |
| 1                | 1    | AAA  | 1    | 50     |
| 2                | 2    | BBB  | 2    | 30     |
| 3                | 3    | CCC  | NULL | NULL   |
| 4                | NULL | NULL | 4    | 40     |

# Outer join where null

OUTER JOIN - WHERE NULL



```
SELECT *  
FROM Table1 t1  
FULL OUTER JOIN Table2 t2  
    ON t1.Col1 = t2.Col1  
WHERE t1.Col1 IS NULL  
    OR t2.Col1 IS NULL
```